# 2026-05-31 - KXTJ3-1057 Accelerometer with CBOR Encoding over USB

## Goal

Extend the KXTJ3-1057 driver from Day 15 to encode accelerometer readings as CBOR and transmit them over USB CDC-ACM. Add shell commands to control the output format and a Python script to receive, decode, and display the data on a PC.

## What you will learn

- How to combine zcbor encoding with real sensor data
- How to design a simple binary protocol using CBOR maps
- How to trigger CBOR output from an interactive shell
- How to write a Python receiver to decode and display the sensor stream

## CBOR frame format

Each transmission is a self-contained CBOR map:

```
{
  "seq":  <uint32>,   sequence number
  "x":    <int32>,    X-axis in mg
  "y":    <int32>,    Y-axis in mg
  "z":    <int32>,    Z-axis in mg
  "rng":  <uint8>     current range in g (2, 4, 8, or 16)
}
```

The frame is followed by a newline (`0x0A`) as a simple delimiter so the Python receiver can use `readline()` to receive complete packets.

### Encoded size

A typical frame encodes to ~25–30 bytes — well within a single USB packet.

## Shell commands

| Command | Description |
|---|---|
| `accel whoami` | Verify WHO_AM_I register |
| `accel read` | Print one reading as plain text |
| `accel cbor` | Send one CBOR frame over USB |

| Command | Description |
|---|---|
| `accel stream <n>` | Send n CBOR frames at 200 ms intervals |
| `accel range <g>` | Set measurement range (2/4/8/16) |

# Python decoder (`tools/decode_accel.py`)

The script connects to the USB serial port, reads CBOR frames, and prints a formatted table. Optional `--plot` flag renders a live scrolling graph using matplotlib.

```
# Install dependencies
pip install pyserial cbor2 matplotlib

# Basic usage
python3 tools/decode_accel.py --port /dev/ttyACM0

# With live plot
python3 tools/decode_accel.py --port /dev/ttyACM0 --plot
```

### Example output

```
SEQ    X (mg)   Y (mg)   Z (mg)   Range
----  --------  -------  -------  -----
   0       -12       34      998    ±2g
   1       -13       33      997    ±2g
   2       -11       35      999    ±2g
```

# Why this matters

Sending structured binary data (CBOR) instead of plain text strings is essential for production firmware — it is compact, self-describing, and easy to parse on any platform. This pattern is used in CoAP, SenML, LwM2M, and Bluetooth GATT descriptors.

# Practice tasks

1. Flash and run `accel cbor` — confirm the Python script decodes correctly.
2. Run `accel stream 50` and observe the live plot.
3. Shake the board and confirm all three axes respond.
4. Switch to `accel range 4` mid-stream and confirm the mg values change scale.

## Example folder

See `src/2026-05-31_KXTJ3_CBOR_USB` for the complete firmware and Python decoder.

## Next topic

Future topics: BLE peripheral advertising sensor data, MCUboot OTA updates, low-power sleep with sensor wake-on-motion.